

Cloud SDK for Scotty: Unified Resumable Upload Protocol

Status: Draft > Review > Current > Needs update > Obsolete

Link: go/cloudsdk-scotty-upload

Authors: Viacheslav Rostovtsev

Last updated: Jul 5, 2026

See also: [Cloud SDK for Scotty: the API team's and the end-user's CUJs](#)

[Cloud SDK for Scotty: Cloud SDK Libraries update requirements](#)

Username	Role	Status	Last change
coryan	Approver	 Pending	Jul 1, 2026
alexghi	Reviewer	 Pending	Jul 6, 2026
atarafamas	Reviewer	 Pending	Jul 14, 2026
betterbrent	Reviewer	 Pending	Jul 1, 2026
blakeli	Reviewer	 Pending	Jul 6, 2026
gabepearhill	Reviewer	 Pending	Jul 6, 2026
hongalex	Reviewer	 Pending	Jul 6, 2026
mzp	Reviewer	 Pending	Jul 6, 2026
partheniou	Reviewer	 Pending	Jul 14, 2026
sdhart	Reviewer	 Pending	Jul 6, 2026
shollyman	Reviewer	 Pending	Jul 14, 2026
suzmue	Reviewer	 Pending	Jul 6, 2026
 Approval Instructions: Please approve or LGTM through the G3 Assist sidebar. For more information, see go/g3a-approvals-reviewing			

Summary

Scotty (go/scotty, go/scotty-http-protocols) is a shared Google service for handling large HTTP/HTTPS uploads and downloads. Scotty acts as a frontend that offloads the complexity of file transfers from individual product backends. Google API teams using Scotty implement a "customer agent" backend that Scotty notifies about transfer progress and completion, allowing the team's application to manage the file and control access. Cloud SDK Libraries are adding support for Scotty functionality.

This document describes the Unified Resumable Upload Protocol from the Cloud SDK point of view. The Unified Resumable Upload protocol is designed and controlled by Google. The protocol description can be found here: [Unified Upload Protocol / Resumable Upload Protocol](#). There is also a smaller overview: [Scotty HTTP Protocols Overview / Unified Resumable Protocol](#). This document is not intended to repeat the protocol specification documents fully. Instead it presents the parts that are important for Client Libraries.

Note on terminology

Throughout this document I will refer to the Unified Resumable Upload protocol as simply "the upload protocol" or "the protocol".

I will talk about the protocol state machine, and generally use a state-machine related terminology. This does not mean that this document prescribes that the protocol should be implemented as a state machine. Rather, the protocol state machine is a concept used to simplify descriptions.

I will also use terms "phase" and "state" somewhat interchangeably. A protocol state machine being "in" a state "X" can be described as a protocol flow having the "X" phase: consisting of the state machine transitioning to state "X", performing some process inside that state (e.g. sending a request, retrying) and then transitioning out of the state "X".

The term "stream" is used for the end-user-provided data object, as a general all-purpose abstraction.

Scope limitation

This document focuses on the protocol. This means it elides certain topics, most importantly:

- Handling of the end-user-provided data object, including the capabilities of the Stream objects, such as Seeking and Rewinding. It is assumed that the data is retrieved when needed.

- Buffering data. The only nod to that concept is passing acknowledgement that one state ("[Transmission](#)") has to have access to the end-user provided data object, while the other ("[Finalizing](#)") has access to a buffer of data that was read from said data object.
 - As a specific case, the buffering ahead of the current chunk is not discussed, so it is assumed that e.g. if the implementation has reached the end of the stream, it is strictly because it has sent everything before that, and is going to send the final chunk.
- Errors arising from reading and buffering data from the end-user provided data object, although technically those are Cat 3 "unretrieable" errors of "any other error" variety and covered by the "[Terminal error](#)" flow.
- Errors occurring in terminal states, e.g. a potential error arising from decoding the result message from json.
- Collecting and verifying [the end-user-provided parameters](#) (such as additional headers). It is assumed that the configuration is already available (so no discussion of Scotty helpers).
- Anything related to the end-user-provided callback, including the timing of calling it.
- Calculation of checksum, as opposed to sending of checksum, which is discussed in the "[Finalizing](#)" state.

In short, a lot of implementation-specific or adjacent concerns are not discussed, purely to keep this document to a readable size.

The end-user-specified parameters

Due to the document phase-focused structure, the end-user-specified parameters are split between phases. Short index:

- Discussed in the context of the "Starting" phase: [the headers](#), [the request message](#).
- Discussed in the context of the "Transmission" phase: [the chunk size](#), [the chunk timeout](#).
- Discussed in the context of the "Finalizing" phase: [the checksum type](#)
- Discussed in the context of phase structure: [the local deadline](#).
- Discussed separately, as the global concern: [the global deadline](#).

The global deadline

The library invocation of the protocol session must have a global deadline. This deadline covers one attempt at Scotty Upload, consisting of multiple requests. Those requests might be potentially retried. This global deadline is separate from:

- The "attempt deadline", which is typically given to an underlying HTTP library, as a parameter for making *one specific HTTP request*.

- The [local deadline](#), which is applied over the transient retry cycle for any given HTTP request (and therefore potentially covers multiple retry attempts of the same request).
- A deadline for Scotty server-side timeout for the upload session, which can be configured very large (up to 3 days) and allows the upload to be interrupted and continued later.

The global deadline is configured as is usual for the Gapic-generated libraries:

- If the end-user is specified a deadline, that value takes precedence
- Then, if the API team has specified a deadline in grpc config, that value is taken (if the language implementation supports this).
- Then, if neither the end-user nor the API team has specified a deadline, the Library configures a sensible default (e.g. 10 minutes).

The sensible default might appear too large for the data transmission. Typical upload size is hundreds of megabytes, and those don't take 10 minutes to upload. However we configure this amount of time by default the libraries for even smaller calls because it has to account for server-side delays and retrying with backoff.

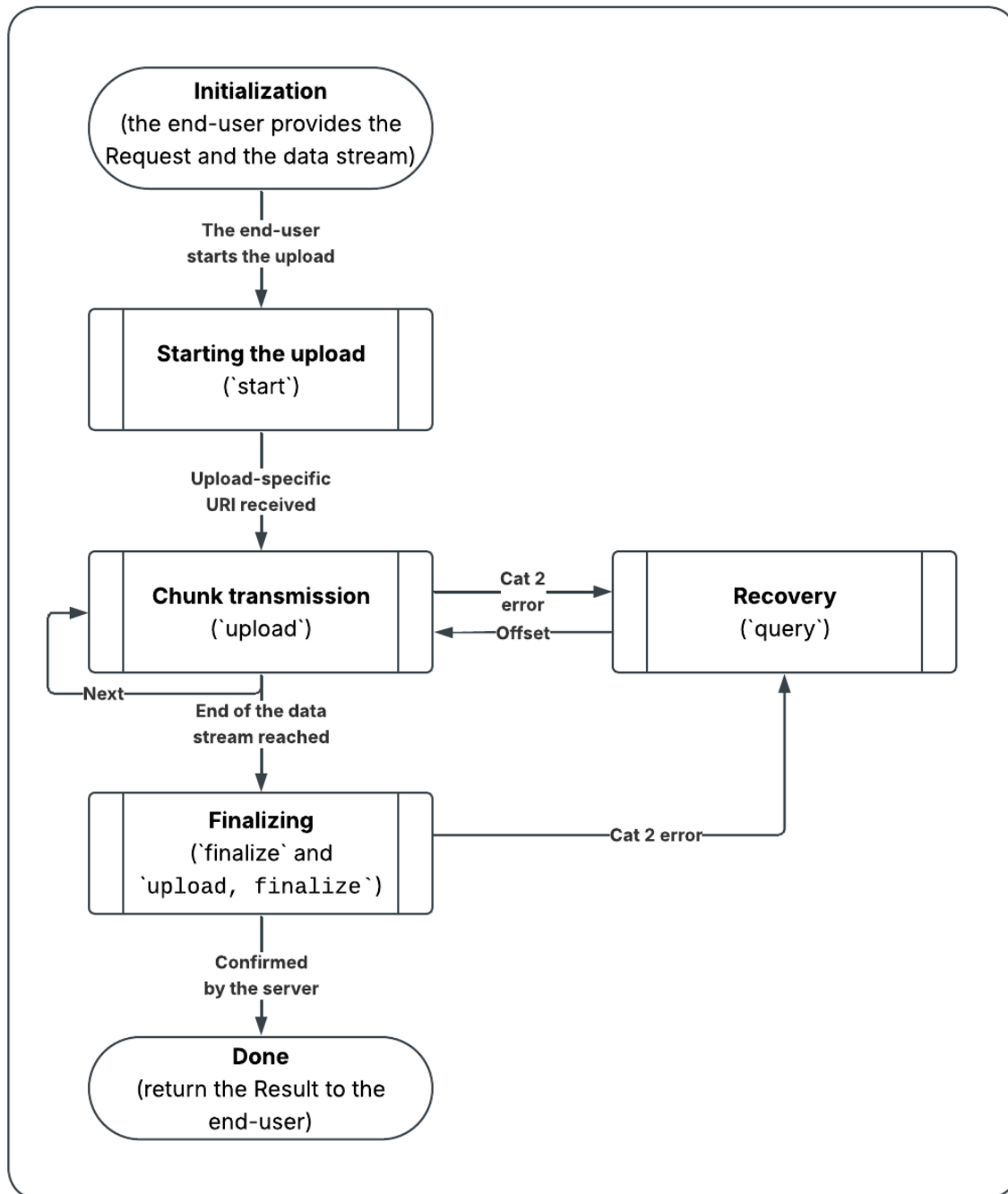
The only addition for Scotty is as following: the Client Libraries MAY revise the default deadline upward if:

- (1) The end-user and the API team has not provided any values for the deadline (and so we have to use the default), and
- (2) The end-user has provided the upload size (which is optional).

In that case the Client Libraries MAY use a sensible default for the upload speed (e.g. 100 Mb/s) and increase the deadline if the size divided by the upload speed default is greater than the default deadline. This is optional. We ultimately rely on the end-users to set the appropriate deadline for their uploads.

The main "Success" flow of the upload protocol

Main flow for the Resumable Upload protocol



The typical successful protocol flow will consist of a "[Starting](#)" phase followed by one or more "[Chunk transmission](#)" (aka just "Transmission") phases, followed by a "[Finalizing](#)" phase. The "[Finalizing](#)" phase will typically send a combined `upload, finalize` command so it will upload the last chunk and finalize the upload in the same request. In rare cases this phase will issue [a separate finalize command](#).

Both "Transmission" and "Finalizing" phases can transition to the "[Recovery](#)" phase if they encounter Cat 2 errors, but the control is always [returned to the "Transmission"](#) phase.

Categories of errors

This document refers to 3 categories of errors: Retriable (Cat 1), Recoverable (Cat 2), and Unrecoverable (Cat 3). The list comes from the protocol description: [section on retrying errors, addendum](#) (I have confirmed with the Scotty team that this is still the best error reference).

Category	Description	Example Codes	Action
Category 1	Retriable Without Modification (Transient)	<code>408, 429, 500, 502, 503, 504</code> , TCP/Socket Timeout	Retry the request
Category 2	Retriable With Modification (only applicable in the "Transmitting" and "Finalizing" states.	<code>400, 412, 416</code> , missing <code>X-Goog-Upload-Status</code> header	Transition to the " Recovery " state
Category 3	Unrecoverable, Unretrieable (Fatal)	Any other error, including but not limited to: <code>401, 403, 404</code> , retrieable errors that have exhausted either the local deadline or attempt count	Bubble up to the end-user

Notable errors: TCP/Socket timeout

This kind of error would typically show up as an exception, specific to the language and the underlying HTTP library. It is retrieable. If the server did receive and has committed part of the data, it will reply with a Cat 2 recoverable error when a retry attempt is made.

Notable errors: missing `X-Goog-Upload-Status` header

Protocol documentation makes a special note that if this header is missing, the client should transition to recovery, even if the response status is 200. For any other state except

"Transmitting" or "Finalizing" this is an unrecoverable error, same as any other "malformed reply" errors.

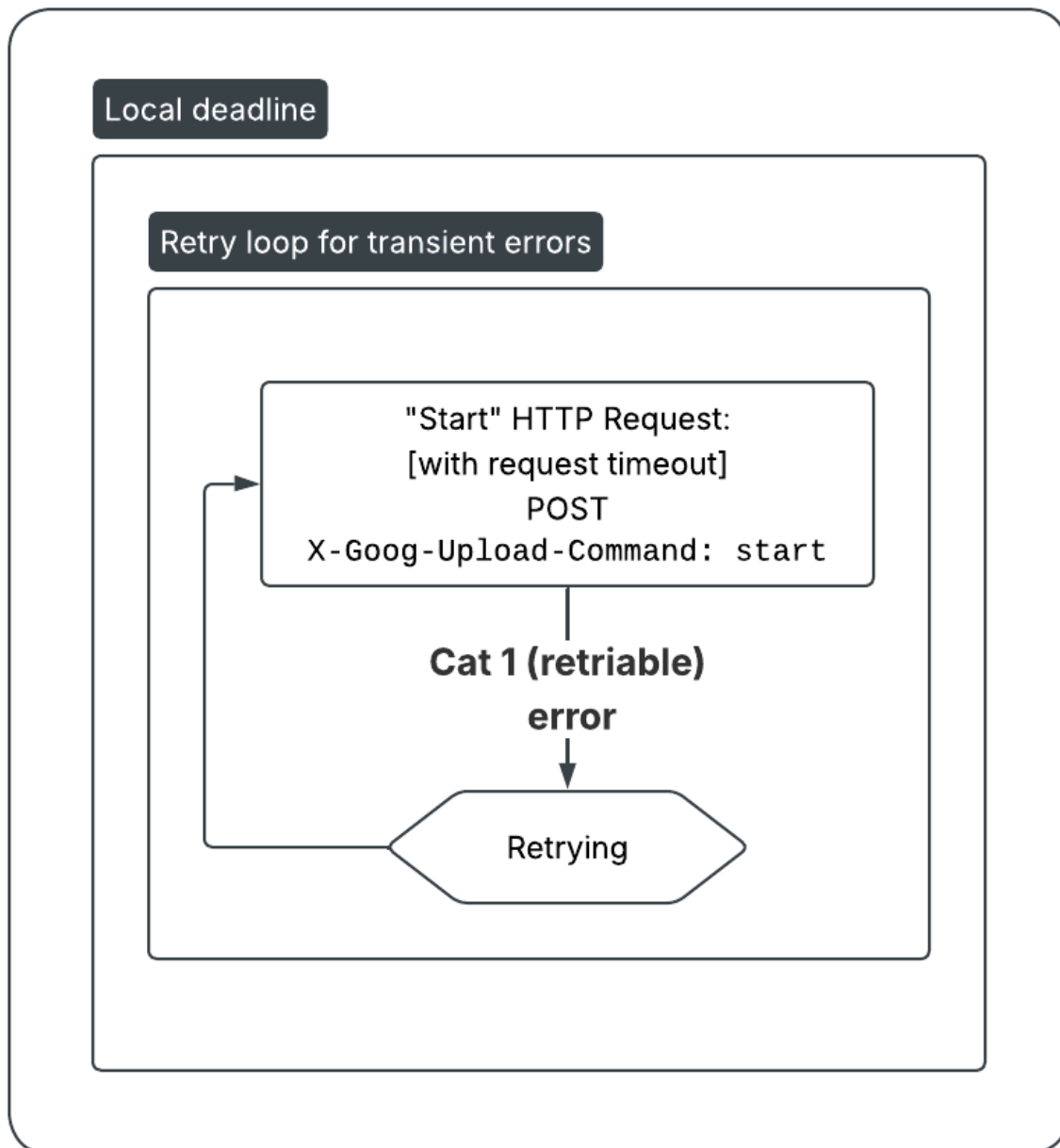
Notable errors: HTTP 308

In previous versions of the protocol Scotty could return HTTP 308 as a resumable error. This behavior [is removed](#) from the Unified Resumable Upload protocol, so 308 is no longer a Cat2 error.

Inner structure of a phase

All the phases in the diagram above contain a retry with a deadline check. Here is a structure of a phase, using without loss of generality the "Starting" phase as an example:

Detailization of the "Start" state process



Every non-terminal state is associated with a protocol command that is sent as a HTTP request to the server-side endpoint. For the "Starting" state the command is `start`. Every HTTP request must be wrapped in a retry loop with a local deadline.

The retry loop must retry only retriable ("Cat 1") errors. Retries should be made with backoff, with sensible default parameters (same as Libraries typically have for unary requests).

A retrievable error that has exhausted either the retry timeout or attempt count becomes a [terminal error](#).

The local deadline

The local deadline is a deadline given to the transient error retry functionality in GAX, possibly as a timeout (which is the length of a deadline). Note that the local deadline applies to the transient retry cycle as a whole.

The local deadline is distinct from the "attempt deadline", which is typically given to an underlying HTTP library, as a parameter for making *one specific HTTP request*, although they may have the same value.

The local deadline is distinct from the [global deadline](#) specified for the upload attempt as a whole, although they can be equal, especially towards the end of the long upload session.

The recommendation from the BQ team was to set the length of the local deadline to be **at most the length of the half** of the initial timeout. E.g. if the end-user has specified the initial deadline to be 10 minutes from the start of the call, then the initial timeout is 10 minutes, and the local deadlines should have a length of at most 5 minutes from the beginning of the call. This, however, is left to the language implementations.

A special form of the local deadline used only for the chunk upload commands is called the chunk upload timeout or ["chunk timeout"](#). It is optional for implementations to allow the end-users to specify it.

"Starting" state

The command for the "Starting" phase is `start`. It is very different from other Scotty commands.

Different "Starting" endpoint and the upload prefix

The HTTP request containing the `start` command will be sent to a different endpoint from the other commands. The "Starting" endpoint is constructed from the endpoint specified in `google.api.http` annotation with the addition of the scotty upload prefix, which is typically `resumable/upload`. The API team members will provide the upload prefix when opting-in to the Scotty generation, as part of the configuration in the libraries section of the services config.

Example:

```
None
# In the API protos
rpc CreateYouTubeVideoUpload(CreateYouTubeVideoUploadRequest)
```

```
    returns (CreateYouTubeVideoUploadResponse) {
    option (google.api.http) = {
        post: "/v24/customers/{customer_id=*/youtubeVideoUploads:create"
    }
}

# In the libraries generation configuration and in the library code
Configured upload prefix: `/resumable/upload`.

# At runtime:
"Starting" url is calculated to be
"https://{host}/resumable/upload/v24/customers/foo/youtubeVideoUploads:create"
instead of
"https://{host}/v24/customers/foo/youtubeVideoUploads:create"
```

The end-user provided headers

The end-user must be able to specify headers that will be sent with the `start` command only. An example of such a header is the Google Ads developer key header `developer-token`.

The RequestMessage as the body

The request message of the Scotty RPC will be sent as a body of the `start` command (serialized in JSON). It will typically contain the upload data's semantic metadata such as the video title.

The upload URL

The response to the `start` command will contain the Scotty upload URL which contains the Scotty upload ID. The upload ID must be surfaced to the end-user. The upload URL can be configured (in the Scotty server configuration) to be completely arbitrary, so we cannot guarantee that we can parse the upload ID. The URL will have to be surfaced in its entirety.

The server-specified chunk granularity and the chunk size selection.

In response to the `start` command, the server **might** specify a required chunk granularity with `X-Goog-Upload-Chunk-Granularity` header, e.g.
`X-Goog-Upload-Chunk-Granularity: 50`. The client must always upload in the multiples of specified value, typically by changing the [end-user-provided chunk size](#) to be a closest smaller multiple of the server-specified granularity to the chunk size. It does not matter for the purpose of this calculation whether the chunk size is provided by the end-user or set to a sensible default.

If the chunk granularity is not specified, the chunk size is used unchanged.

Successful start of the upload (`X-Goog-Upload-Status: active`)

In response to the successful `start` command the server sends `X-Goog-Upload-Status: active` in the headers. This header should always be present in server responses during the protocol exchange, until it changes to `X-Goog-Upload-Status: final` after the successful "Finalizing" phase.

The absence of this header is an error: recoverable for "Uploading" and "Finalizing" state, and retrievable for all other states.

Note, that there are two other possible states for this header:

- Receiving a response with `X-Goog-Upload-Status: final` in a response with a non-200 HTTP code is a rejection. [Rejection](#) is a terminal state.
- It is expected that the value of the header will be `X-Goog-Upload-Status: cancelled` after a successful [Cancellation](#).

"Start" state transitions

After the successful `start` command, the protocol transitions to the `Chunk transmission` state.

In response to errors, the `start` command can only be retried. Since the upload session has not been created, there is no upload URL and the [recovery](#) is not possible. Similarly, the end-user cannot `cancel` when the `start` command is ongoing, so the only other transitions possible are to the terminal ["Rejected"](#) or ["Error"](#) states.

"Transmission" state

The command for the "Transmission" phase is `upload`. During a successful upload session this state repeats until all the chunks are successfully uploaded (the end-user provided data stream reaches EOF).

The end-user-provided chunk size

The client must provide the end-user with the ability to specify a chunk size. Then in the "Transmission" phase, the protocol implementation buffers and uploads data in chunks of that size. Note that the end-user-provided chunk size can be adjusted down to be a multiple of [server-specified chunk granularity](#). It must NOT be adjusted up, because presumably it is chosen by the end-user to e.g. match their networking or storage limitations.

The buffering, and its impact on the implementation is [out of scope](#) of this protocol-focused document. It however has an impact on the [transition tables](#), where the transmission phase is split into "reading from the stream" (aka "buffering") and "sending" sub-phases.

If the end-user has not specified a chunk size, the client library must use a sensible default.

The end-user-provided chunk timeout

A language implementation MAY allow the end-user to specify a timeout specifically for the chunk upload, alongside the chunk size. This local "chunk timeout" is a form of a local deadline. If provided, "chunk timeout" must be used to calculate the local deadline for any given chunk upload call, including the "Finalizing" phase's [upload, finalize](#) command. It can optionally (for the sake of simplifying the implementation) be used with the "Finalizing" phase's [finalize](#) command.

Transition to the "Recovery" state

In this phase the protocol implementation can encounter recoverable (aka "Category 2", aka "retriable with modification") errors. The main example of these is [HTTP 416 Range not satisfiable](#). These errors indicate a disagreement between the server and the client on the upload offset. When such an error occurs the protocol implementation should transition to the ["Recovery"](#) phase.

Transition back to the "Transmission" state from the "Recovery" state

In most cases when the state is returned to "Transmission" from "Recovery" we should expect that the offset lies on the border of or within the current chunk. In which case the implementation should read enough data so that the buffer is up to chunk size. It should not try to send the remainder of the buffer as-is, since this might run into granularity issues, if the server has specified a granularity size.

Rewinding the stream in response to recovery

It is possible that, during the recovery, the server replies with an offset that is lower than the offset of the data currently in the buffer of the "Transmission" state. Theoretically if the server has discovered that the data is lost completely, it might return [0](#), requiring the data to be sent anew. In this case we'll have to rewind the stream back to the offset returned from the ["Recovery"](#).

There are several operating concerns here:

First, rewinding the end-user-provided data stream might lead to special errors, if the stream is non-seekable.

Second, if the end-user has specified the chunk size we should not send the amount of data greater than that. This typically means that after rewinding the stream the current buffers should be discarded.

Transition to the finalizing state after recovery

Another thing that can happen is that the stream has reached EOF, in which case the buffer should be aligned to the server-specified offset and the protocol should transition to ["Finalizing"](#) as normal.

Transition to the Finalizing state

This transition does not occur after analyzing the server response to the HTTP request. Instead it occurs when reading from the end-user provided data stream and reaching the end of it. When that happens typically there will be some amount of data just read, but it might happen that there is no data. It makes no difference to this state transition, the data or lack thereof is passed along.

"Finalizing" state

The command for the "Finalizing" phase is `finalize`.

The combined `upload, finalize` command.

If there is no data left over from the ["Transmission"](#), the [body is sent empty with just the `finalize` command](#). In most cases though, there will be some data, typically not a multiple of a chunk size, that needs to be sent along. That data must be sent with `upload, finalize` combined command and not with the `upload` command. There are two concerns here:

- First, if a server has [chosen a granularity](#) the data of the wrong size will be rejected if sent without the `finalize`.
- Second, sending a combined command saves a roundtrip compared to sending two commands.

For these reasons we transition to the "Finalizing" state on end-of-stream instead of first sending the data in the ["Transmission"](#) state.

Successful completion of the upload (`X-Goog-Upload-Status: final`)

After receiving the `finalize` command or the combined `upload, finalize` command, the server acknowledges the successful completion of the upload session by sending a response with the HTTP 200 code and the `X-Goog-Upload-Status: final` header. Only at that point the upload is considered successful.

NB: if the `X-Goog-Upload-Status: final` header is sent with any other status code it's a [Rejection](#) instead.

Sending the checksum header

If the end-user has indicated that the checksum should be sent, the protocol implementation must calculate it when reading from the stream (out of scope of protocol discussion). The checksum is sent with the `finalize` or with the `upload, finalize` command in the `X-Goog-Upload-Header-X-Goog-Hash` header.

There are two possible options for checksum: `md5` and `crc32c`, and the end-user specifies which one to send.

The format for the header is: `X-Goog-Upload-Header-X-Goog-Hash: <checksum_code>=<checksum_value>`, e.g.:

`X-Goog-Upload-Header-X-Goog-Hash: crc32c=MTg0Nzg2NTgx.`

Transition to the Recovery state

In this phase, just like in the ["Transmission"](#) phase the protocol implementation can still encounter recoverable (aka "Category 2", aka "retriable with modification") errors. While these errors should not occur in response to the `finalize` command, it does not need a special implementation. If Cat 2 errors occur whether for `finalize` or `upload, finalize`, the protocol implementation should make a state transition to ["Recovery"](#).

Note that from the ["Recovery"](#) phase the protocol will transition to the ["Transmission"](#) state. This is to avoid special handling if the stream needs to be rewound. In most cases it is expected that the recovery offset will be within the existing data buffers, and the protocol will transition from the ["Transmission"](#) state to "Finalizing" state immediately.

The `finalize` command

Note that typically [the upload, finalize combined command](#) will be used.

However there are two sets of circumstances where a single `finalize` command is issued:

- (1) The end-user-provided data object was an exact multiple of the selected chunk size. After the final chunk of the data was transmitted, the next stream read returned 0 bytes of data and an end-of-stream indicator.
- (2) There was a recoverable error during the "Finalizing" phase. After the recovery, the server-specified offset was at the exact end of the end-user-provided data object (meaning all the data was successfully committed by the server before the recoverable error was returned).

"Recovery" state

The command for the "Recovery" state is `query`. This command can technically be issued at any time, and in parallel with any other command. However we should not expose this functionality to the end-user; instead the command should only be issued in the "Recovery" state.

Reaching the "Recovery" state

This state should be transitioned to when a recoverable error is encountered during the "Transmission" or "Finalizing" state. The recoverable errors are also known as the Cat 2 errors, or errors retrievable with modification. The purpose of the "Recovery" state is to obtain the information for this modification – the offset of the data that the server has committed.

The "Recovery" state always transitions (with said offset) back to the ["Transmission"](#) state which can rewind the stream and read additional data if needed.

Why have the "Recovery" state

Recovery state is a part of the protocol specification. If a server error indicates chunk misalignment a blind retry is only going to be met with the same error. And since the server does not indicate in an error what the correct offset is we should issue the `query` command – which is the "Recovery" state.

Why have retries when there is the "Recovery" state

For the `upload` command in the "Transmission" state and the `upload, finalize` command in the "Finalizing" state it is possible to treat every retrievable error as recoverable instead and issue a `query` command before continuing. I think this is excessive. Certain errors like `503 Unavailable` and `504 Gateway timeout` typically come from GFE and do not require any request modification (and Scotty documentation in the [table here](#) adds `408 Request Timeout` and `429 Too many requests` to that list). I don't think we will achieve some large simplification benefit by avoiding retries either since the retries are handled by the existing retry primitives in the same Gax library.

If a server committed a chunk of data before the error happened it will reply to a retry request with a recoverable error and then we can enter recovery so the retry is not going to put the protocol implementation in a terminal state either.

Secondary ("Unsuccessful") flows

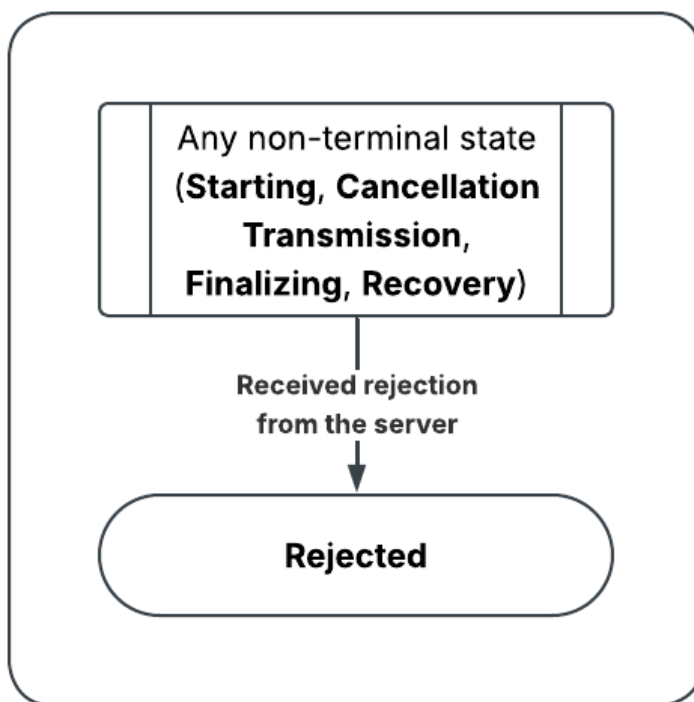
The above documentation covered the states, commands, and transitions during the successful upload session, which might include a recovery. There are, however, a number of secondary

flows that lead to different terminal outcomes and will not occur during the successful upload. These are: "[Rejection](#)", "[Cancellation](#)", and the "[Terminal error](#)" flows, covered below.

Rejection flow

For any command, the server may return the header `X-Goog-Upload-Status: final` in a response with a non-200 HTTP code. This means that the server has rejected the upload. The reason for this will be given in the body of the response and should be raised like a typical server error to the end-user.

Secondary "Rejection" flow

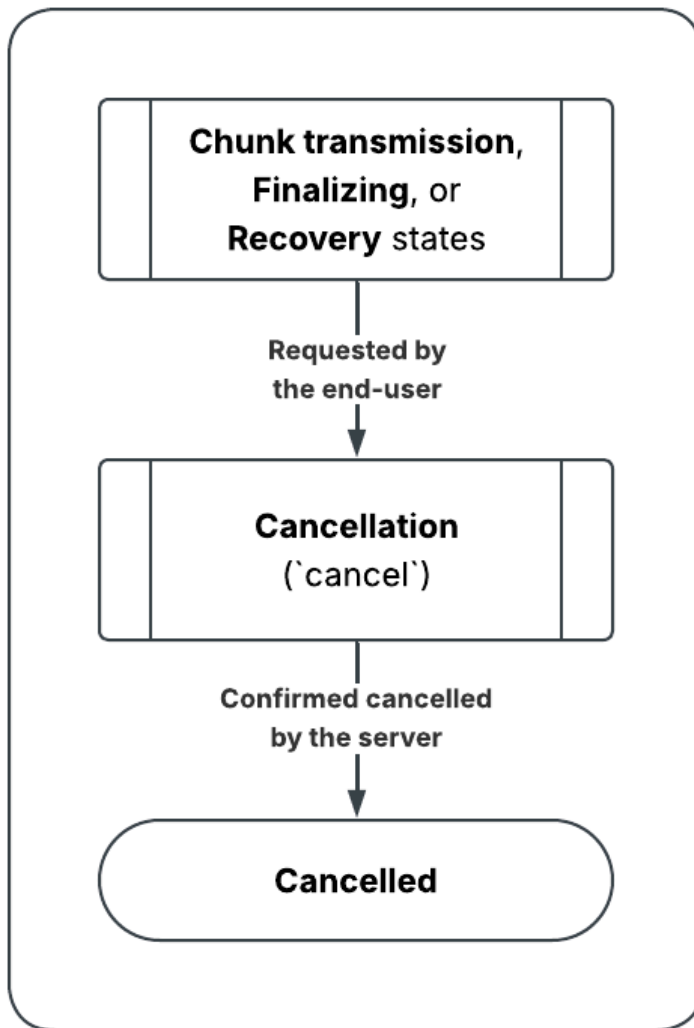


Note that this flow does not have any new non-terminal states.

Cancellation flow

At any point after the start of the upload and before the upload is finalized, the end-user should be able to request a cancellation. The mechanism of that is language-specific; one proposed option is to utilize the callback function that would notify the end-user of the progress.

This flow has a special State: Cancellation.

Secondary "Cancellation" flow**"Cancellation" state**

The command associated with the "Cancellation" state is `cancel`. This command must be retried if it fails with a Cat 1 ("retriable") error. The successful cancellation of the upload will result in the server sending the response with the `X-Goog-Upload-Status: cancelled`, after which the end-user must be notified. The mechanism is left for the language-specific implementation.

Not cancelling during cancellation



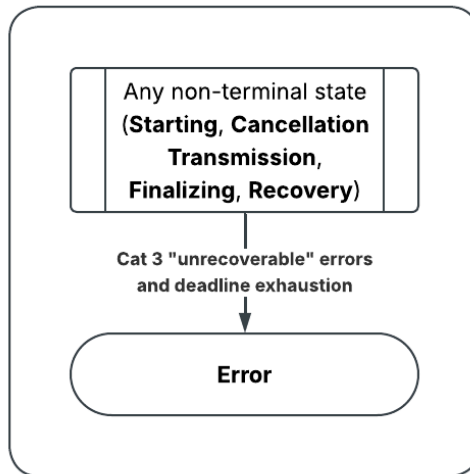
The protocol implementation should not give the end-user an ability to request cancellation during the "Cancellation" phase and must not attempt to make a cancellation request while another cancellation request is in progress.

Terminal error flow

Certain errors are non-retriable and non-recoverable. This includes:

- Being in a recoverable state (e.g. "Transmission") and receiving an error that is not retriable or recoverable
- Being in a non-recoverable state (e.g. "Starting") and receiving an error that is not retriable
- Retrying a retriable error, and exhausting either the local deadline or attempt count
- Exhausting a global deadline
- The server indicates during recovery that the implementation should rewind a stream, and the stream is not-rewindable (implementation concern, but it does come up).

Secondary "Terminal error" flow



Note that this flow does not have any new non-terminal states.

The protocol state transition tables

This table includes states that are important **for the implementation**. Specifically the "Transmission" state and the "Finalizing" state have been split into sub-states for reading data and sending data. This document is describing a protocol and as such does not put reading data from the stream into focus. I did not however want to have two mostly similar state transition tables, so the implementation-scoped table had to appear in the protocol-scoped document.

The protocol state transition table: Success flow

From state		Event	To state	
Terminal	Non-terminal		Non-terminal	Terminal
Initializing		`start_upload` called	Starting	

	Starting	Response: 200, X-Goog-Upload-Status: active	Transmission Reading from stream	
	Transmission Reading from stream	Chunk-size data read, no EOF	Transmission Sending	
	Transmission Reading from stream	Any size data read, EOF	Finalizing Reading from buffer	
	Transmission Sending	Recoverable error	Recovery From Transmission	
	Transmission Sending	Response: 200, X-Goog-Upload-Status: active	Transmission Reading from stream	
	Finalizing Reading from buffer	Offset within buffer, Data needs to be sent	Finalizing Sending with upload	
	Finalizing Reading from buffer	Offset within buffer, No data to send	Finalizing Sending finalize	
	Finalizing Sending with upload	Recoverable error	Recovery	
	Finalizing Sending with upload	Response: 200, X-Goog-Upload-Status: final		Success
	Finalizing Sending finalize	Recoverable error	Recovery	
	Finalizing Sending finalize	Response: 200, X-Goog-Upload-Status: final		Success
	Recovery	Response: 200, X-Goog-Upload-Status: active, Offset	Transmission Reading from stream	

The protocol state transition table: Unsuccessful flows

From state		Event	To state	
Terminal	Non-terminal		Non-terminal	Terminal
	Transmission Sending	End-user event: 'cancel'	Cancelling	
	Finalizing Sending with upload	End-user event: 'cancel'	Cancelling	
	Finalizing Sending finalize	End-user event: 'cancel'	Cancelling	
	Recovery	End-user event: 'cancel'	Cancelling	
	Cancelling	Response: 200, X-Goog-Upload-Status: cancelled		Cancelled
	Starting	Category 3 "Unrecoverable" errors		Error
	Transmission Reading from stream	Category 3 "Unrecoverable" errors		Error
	Transmission Sending	Category 3 "Unrecoverable" errors		Error
	Finalizing Reading from buffer	Category 3 "Unrecoverable" errors		Error
	Finalizing Sending with upload	Category 3 "Unrecoverable" errors		Error

	Finalizing Sending finalize	Category 3 "Unrecoverable" errors		Error
	Recovery	Category 3 "Unrecoverable" errors		Error
	Cancelling	Category 3 "Unrecoverable" errors		Error
	Starting	Rejection (Response: non-200, X-Goog-Upload-Status: final)		Rejected
	Transmission Sending	Rejection (Response: non-200, X-Goog-Upload-Status: final)		Rejected
	Finalizing Sending with upload	Rejection (Response: non-200, X-Goog-Upload-Status: final)		Rejected
	Finalizing Sending finalize	Rejection (Response: non-200, X-Goog-Upload-Status: final)		Rejected
	Recovery	Rejection (Response: non-200, X-Goog-Upload-Status: final)		Rejected
	Cancelling	Rejection (Response: non-200, X-Goog-Upload-Status: final)		Rejected

Bibliography

1. [Unified Upload Protocol / Resumable Upload Protocol](#)
2. [Scotty HTTP Protocols Overview / Unified Resumable Protocol](#)
3. [Maintaining data integrity in GCS via checksumming](#)

Addendum: Scotty documents

- 📄 Teams requesting Scotty features
- 📄 Cloud SDK for Scotty: the API team's and the end-user's CUJs
- 📄 Cloud SDK for Scotty: Universal Resumable Upload Protocol
- 📄 Cloud SDK for Scotty: Cloud SDK Libraries update requirements